HIGH-PERFORMANCE APPLICATION

With Advanced Architecture

Technical Documentation and Performance Analysis

Prepared by: Antony John Robees

Project Specialization: High-traffic Web Service

Programming Language: Python 3.12.4

Primary Framework: FastAPI

Database: SQLite

ORM: SQLAlchemy

Documentation Version: 1.0

Project Deadline: November 7, 2026

Table of Contents

HIGH-PERFORMANCE APPLICATION **1**
With Advanced Architecture 1
Technical Documentation and Performance Analysis 1
Table of Contents **2**
1. Project Overview **4**
1.1 Introduction 4
1.2 Project Objectives 4
1.3 Project Specialization 4
2. Technology Stack **5**
2.1 Why These Technologies Were Selected 6
3. System Architecture **7**
3.1 Architectural Overview 7
3.2 Architecture Diagram 7
3.3 Request Processing 8
4. Project Structure **8**
4.1 Current Directory Structure 8
4.2 Module Responsibilities 10
5. Database Design and Optimization **11**
5.1 Database Configuration 11
5.2 Product Data Model 12
5.3 Database Query Optimization 12
5.4 Pagination Response 13
5.5 Query Plan Observation 14
6. Caching and Performance Optimization **15**
6.1 Caching Overview 15
6.2 Cache Configuration 15
6.3 Cache Operations 16
6.4 Cache Integration 16
6.5 Cache Invalidation 17
6.6 Thread Safety 17
6.7 Cache Testing 17
7. API Implementation **17**
7.1 API Overview 17
7.2 Available Endpoints 18
7.3 Product Creation 18
7.4 Product Retrieval 19
7.5 Product Updating 19

7.6 Product Deletion....20
7.7 Request Validation....20
8. Concurrency and Performance Management....21
8.1 Concurrency Approach....21
8.2 Cache Synchronization....21
8.3 Load Testing....21
9. Testing and Quality Assurance....22
9.1 Testing Strategy....22
9.2 API Testing....22
9.3 Cache Testing....23
9.4 Test Database Isolation....23
10. Performance Evaluation....24
10.1 Performance Testing Objective....24
10.2 Locust Benchmark Results....25
10.4 Performance Optimization Techniques....27
11. Application Logging and Monitoring....27
11.1 Logging Configuration....27
11.2 Log Organization....28
11.3 Monitoring Approach....28
12. Installation and Execution....29
12.1 Prerequisites....29
12.2 Environment Setup....29
12.3 Running the Application....30
12.4 Running Automated Tests....30
12.5 Running Load Tests....31
13. Scalability Considerations and Future Improvements....31
13.1 Database Scalability....31
13.2 Distributed Caching....32
13.3 Asynchronous Database Operations....32
13.4 Containerization....32
13.5 Orchestration....32
13.6 Advanced Monitoring....33
13.7 Scalability Limitations....33
14. Conclusion....34

1. Project Overview

1.1 Introduction

The High-Performance Application with Advanced Architecture is a Python-based web application developed to explore performance optimization, efficient database operations, caching, API development, and application testing.

The project implements a product management API using FastAPI and SQLAlchemy, with SQLite as its database. It provides essential operations for creating, retrieving, updating, and deleting product records.

To improve response efficiency, the application incorporates in-memory caching, paginated database retrieval, and structured application logging. Its performance is evaluated using Locust, while Pytest is used to verify application functionality and input validation.

The project focuses on understanding how different architectural and optimization techniques influence application performance and how these techniques contribute to building scalable web services.

1.2 Project Objectives

The primary objectives are:

- Develop a RESTful API using FastAPI.
- Implement complete product management operations.
- Optimize database retrieval through pagination and efficient queries.
- Introduce in-memory caching to reduce repeated database access.
- Implement thread-safe cache operations.
- Establish structured application logging.
- Develop automated tests for API endpoints and caching functionality.
- Evaluate application performance through load testing.
- Understand application profiling, concurrency, and scalability considerations.

1.3 Project Specialization

Selected specialization: High-Traffic Web Service

This specialization focuses on designing an API capable of handling repeated requests efficiently while maintaining consistent data retrieval and application behavior.

The project explores performance techniques commonly used in web services, including:

- Database query optimization.
- Pagination.
- In-memory caching.
- Cache invalidation.
- Request validation.
- Load testing.
- Performance monitoring through logs and benchmark measurements.

The current implementation serves as a foundation for understanding high-traffic application architecture. It is not yet a distributed or production-scale deployment.

---

2. Technology Stack

The application uses the following technologies:

Technology	Purpose
Python 3.12	Primary programming language
FastAPI	REST API development
Uvicorn	ASGI application server
SQLite	Relational database
SQLAlchemy	Database ORM and query execution
Pydantic	Request and response validation
cachetools	In-memory TTL caching

threading	Cache synchronization
Pytest	Automated testing
HTTPX	API testing through TestClient
pytest-cov	Test coverage measurement
Locust	Load testing
cProfile	Python application profiling
Python logging	Application event logging

2.1 Why These Technologies Were Selected

FastAPI: Provides a structured approach to building REST APIs, automatic request validation, and interactive API documentation.

SQLite: Offers a lightweight relational database suitable for development and testing without requiring a separate database server.

SQLAlchemy: Simplifies database interaction through ORM models and structured SQL queries.

cachetools: Provides a TTL-based caching mechanism with configurable expiration and storage capacity.

Pytest: Supports automated testing of application functionality and validation behavior.

Locust: Simulates concurrent users and measures API response times and request failures.

Python logging: Records application events to separate log files for easier troubleshooting and observation.

---

3. System Architecture

3.1 Architectural Overview

The application follows a layered architecture in which API endpoints, validation schemas, database models, database configuration, and caching responsibilities are separated into individual modules.

The main components are:

1. API Layer – Handles incoming HTTP requests.
2. Validation Layer – Validates incoming data and structures responses.
3. Business Logic Layer – Performs product management operations.
4. Caching Layer – Stores frequently requested data temporarily.
5. Database Layer – Manages database connections and sessions.
6. Logging Layer – Records application events.

3.2 Architecture Diagram

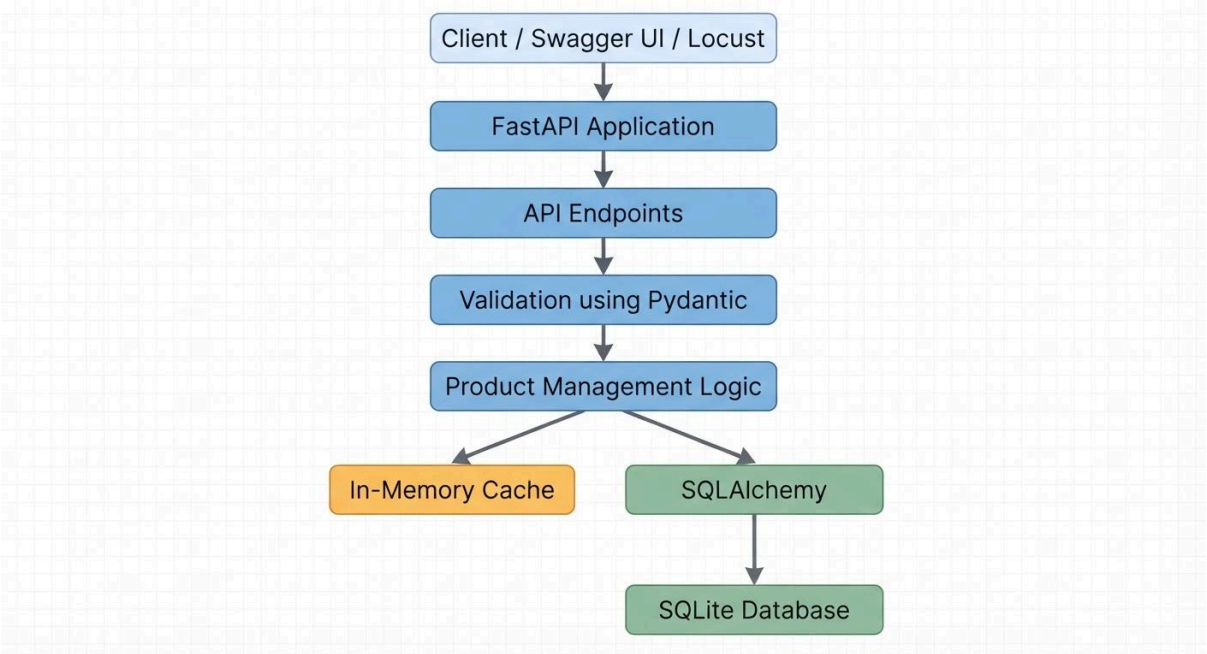


Figure 1: Architecture Diagram

Logging is connected to the application and caching components to record relevant events.

3.3 Request Processing

When a client sends a request, FastAPI receives it and routes it to the corresponding endpoint.

For operations involving product retrieval, the application checks whether the requested information is available in the cache.

If the cached information exists, it is returned without executing the corresponding database retrieval query.

If the information is unavailable or expired, the application retrieves the required records from SQLite through SQLAlchemy.

The response is then validated and returned to the client.

For product creation, updating, and deletion, the application modifies the database and clears the cache to prevent outdated information from being returned.

---

4. Project Structure

The project is organized into modules based on their responsibilities.

4.1 Current Directory Structure

```
None
High-Performance Application/
│
├── configs/
│   ├── __init__.py
│   └── general_logger.py
│
├── data/
│   └── products.db
```

```
|
├── docs/
|   └── screenshots/
|       ├── application_logs.png
|       ├── locust_performance.png
|       ├── pagination.png
|       ├── product_creation.png
|       ├── pytest_results.png
|       └── swagger_ui.png
|
├── logs/
|   ├── main_api.log
|   └── memory_cache.log
|
├── tests/
|   ├── test_cache.py
|   └── test_products.py
|
├── cache.py
├── config.py
├── database.py
├── locustfile.py
```

```
├── main.py
├── models.py
├── requirements.txt
├── schemas.py
└── .gitignore
```

Note: Virtual environments, Python cache directories, and temporary test artifacts are excluded from this representation.

4.2 Module Responsibilities

File / Directory	Responsibility
main.py	FastAPI application and product API endpoints
cache.py	Thread-safe in-memory caching
config.py	Project paths and database URL configuration
database.py	SQLAlchemy engine, session and base configuration
models.py	Product database model
schemas.py	Pydantic request and response schemas
locustfile.py	Locust load-testing configuration

configs/	Shared application configuration
general_logger.py	Logger creation and file handling
tests/	Automated application tests
data/	SQLite database storage
logs/	Application log files
docs/screenshots/	Visual evidence of application functionality
requirements.txt	Python dependency specifications

---

5. Database Design and Optimization

5.1 Database Configuration

SQLite is used as the application's relational database.

The database configuration is managed through `config.py` and `database.py`.

The configuration module establishes the project directory and ensures that the `data` directory exists before constructing the database URL.

The database module creates the SQLAlchemy engine and session factory.

A database session is provided to API endpoints through the `get_db()` dependency. The session is closed after the request finishes.

5.2 Product Data Model

The application currently uses one database model named `Product`.

Database table: `products`

Column	Data Type	Description
product_id	Integer	Primary key
product	VARCHAR(100)	Product name
price	Integer	Product price
stock	Integer	Available stock
stocked_date	DateTime	Product registration date and time

The `product_id` column serves as the primary key.

The `stocked_date` value is generated automatically using the current UTC datetime when a new product is created.

5.3 Database Query Optimization

The application uses SQLAlchemy's `select()` statements to retrieve records.

Several techniques are implemented:

Primary-key retrieval

Individual products are retrieved using their `product_id`. This uses the database's primary-key lookup mechanism.

Pagination

The product listing endpoint supports `skip` and `limit` parameters.

Default configuration:

- Skip: 0
- Limit: 10
- Maximum limit: 50

Pagination prevents the application from retrieving every product record when a client requests only a limited number of results.

Ordered retrieval

Product records are retrieved in ascending `product_id` order to provide consistent pagination.

Total record counting

The application uses SQL `COUNT(*)` through SQLAlchemy to calculate the total number of product records.

This information is included in the paginated response.

5.4 Pagination Response

The product listing endpoint returns:

- `products`: Retrieved product records.
- `skip`: Current pagination offset.
- `limit`: Maximum requested page size.
- `total`: Total number of products.
- `has_more`: Indicates whether additional records are available.

This provides clients with sufficient information to navigate through product collections.

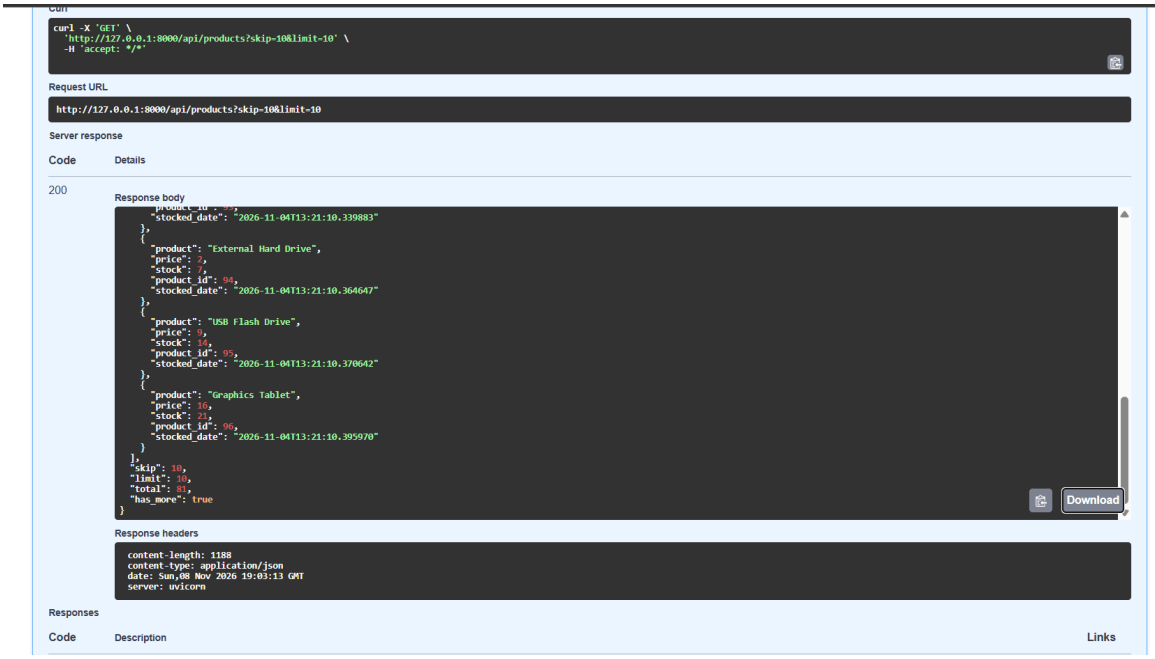


Figure 2: Product pagination response

5.5 Query Plan Observation

SQLite query-plan analysis was performed to understand how the database executes product retrieval queries.

The observed queries included:

- Product retrieval using the integer primary key.
- Ordered product retrieval using `product_id`.
- Total record counting.

The primary-key lookup uses the SQLite integer primary-key mechanism. The ordered listing and count operations were also examined.

No additional secondary indexes have been implemented in the current version.

---

6. Caching and Performance Optimization

6.1 Caching Overview

Caching is implemented to reduce repeated database retrieval operations.

The application uses `cachetools.TTLCache` to store frequently requested product information in application memory.

The cache is managed through the `MemoryCache` class in `cache.py`.

6.2 Cache Configuration

The current cache configuration is:

Setting	Value
Cache implementation	TTLCache
Maximum capacity	1,000 entries
Time to live	30 seconds
Synchronization	threading.Lock
Storage	Application memory

The TTL mechanism automatically expires entries after their configured lifetime.

The maximum capacity restricts how many entries can be stored.

6.3 Cache Operations

The `MemoryCache` class provides four operations:

get_cache()

Retrieves a cached value using its key. Returns `None` when the entry is missing or expired.

set_cache()

Stores a value in the cache.

delete_cache()

Removes an individual cache entry when required.

clear_cache()

Removes all entries currently stored in the cache.

All four operations are protected by a threading lock.

6.4 Cache Integration

Caching is integrated into two retrieval endpoints.

Product listing

The application uses the following cache-key pattern:

```
products_{skip}_{limit}
```

This allows different pagination requests to have separate cache entries.

Individual product retrieval

The application uses:

```
product_{product_id}
```

This allows individual product records to be cached separately.

6.5 Cache Invalidation

When a product is created, updated, or deleted, the application clears the entire cache.

This prevents previously cached product information from being returned after database modifications.

Although clearing the complete cache is straightforward, it can remove entries that are still valid.

More selective cache invalidation could be considered as a future optimization.

6.6 Thread Safety

The application uses Python's `threading.Lock` to protect cache operations.

This prevents multiple threads from simultaneously modifying the cache through the exposed cache methods.

The lock provides synchronization within the running application process. It does not provide distributed caching or synchronization between multiple application instances.

6.7 Cache Testing

The caching module is tested through three primary test cases:

1. Setting and retrieving cached values.
2. Deleting individual cached entries.
3. Clearing all cached entries.

The tests verify that missing entries return `None` and that stored values can be retrieved correctly.

---

7. API Implementation

7.1 API Overview

The application provides product management functionality through RESTful HTTP endpoints.

The endpoints are implemented in `main.py`.

7.2 Available Endpoints

HTTP Method	Endpoint	Functionality
GET	/	Root application response
GET	/api/products	Retrieve paginated products
POST	/api/products	Create a product
GET	/api/products/{product_id}	Retrieve an individual product
PATCH	/api/products/{product_id}	Update an existing product
DELETE	/api/products/{product_id}	Delete a product

7.3 Product Creation

The POST endpoint accepts product information through a JSON request body.

The incoming data is validated using the `ProductCreate` schema.

After validation, the application:

1. Creates a SQLAlchemy Product instance.
2. Adds the record to the database session.
3. Commits the transaction.
4. Refreshes the instance to retrieve database-generated values.
5. Clears the cache.
6. Returns the created product.

Successful creation returns HTTP 201.

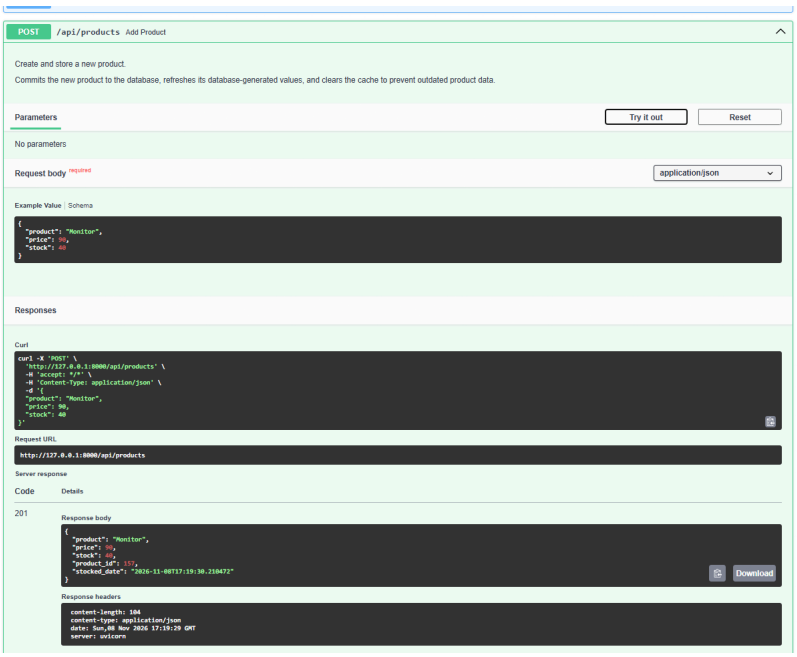


Figure 3 Product creation through Swagger UI

7.4 Product Retrieval

The application supports two retrieval operations.

Retrieve all products

The listing endpoint supports pagination and caching.

Retrieve an individual product

The application first checks the cache. If the requested product is not cached, it queries the database using the product's primary key.

If the product does not exist, the API returns HTTP 404.

7.5 Product Updating

The PATCH endpoint supports partial updates.

Only the fields supplied by the client are included in the update operation through Pydantic's `model_dump(exclude_unset=True)`.

The application then updates the corresponding model attributes, commits the transaction, refreshes the record, and clears the cache.

This approach allows clients to modify selected product fields without submitting the entire product record.

7.6 Product Deletion

The DELETE endpoint retrieves the product using its primary key.

If the product exists, it is deleted and the transaction is committed.

The cache is then cleared.

Successful deletion returns HTTP 204. If the product does not exist, the endpoint returns HTTP 404.

7.7 Request Validation

Pydantic schemas are used to validate incoming product information and structure API responses.

FastAPI automatically generates validation responses when incoming requests do not satisfy the defined schema requirements.

The application tests include invalid product names, invalid prices, and negative stock values.

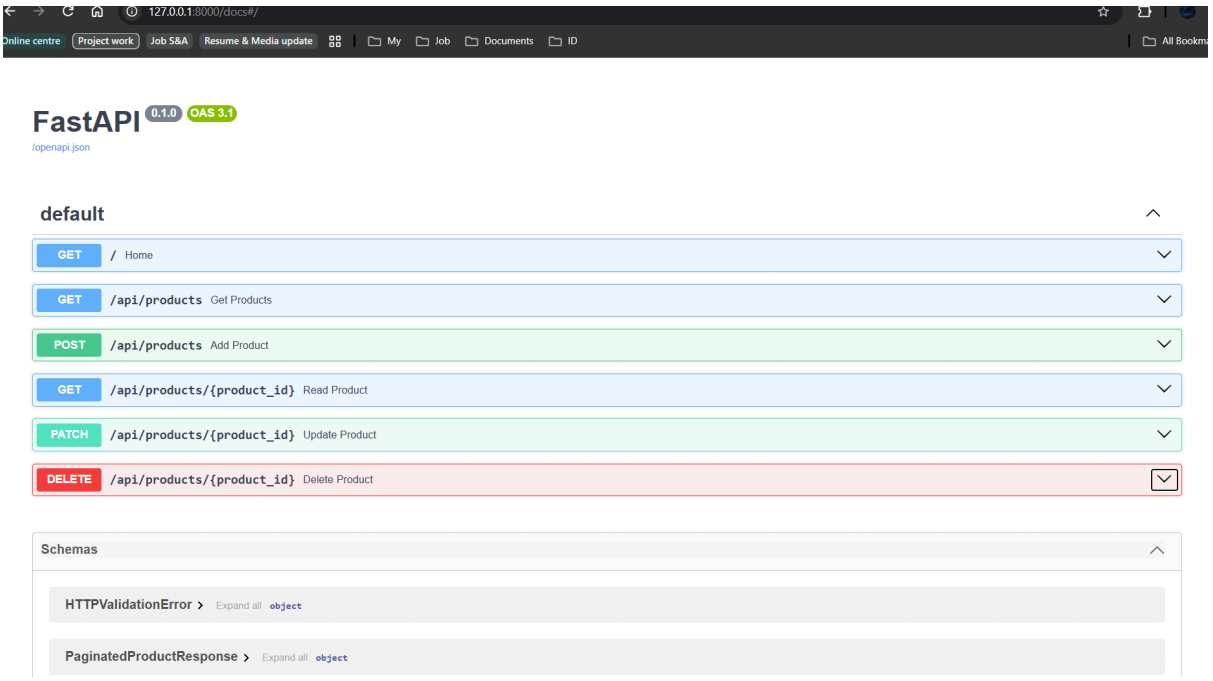


Figure 4 Interactive API documentation

8. Concurrency and Performance Management

8.1 Concurrency Approach

The application is built using FastAPI and synchronous SQLAlchemy database operations.

The API endpoint functions are defined using regular Python `def` functions rather than `async def`.

FastAPI supports synchronous endpoint execution through its thread-pool execution model.

This allows synchronous operations to be handled without blocking the event loop in the same way as directly executing blocking work inside an asynchronous endpoint.

The current implementation does not contain explicitly implemented asynchronous database operations or parallel processing algorithms.

8.2 Cache Synchronization

The caching module uses `threading.Lock` to protect shared cache operations.

This is important when multiple requests access the same application-level cache.

The synchronization mechanism protects individual cache operations. It does not make an entire sequence of cache lookup, database retrieval, and cache insertion atomic.

8.3 Load Testing

Locust is used to simulate concurrent API users.

The load-testing configuration is maintained in `locustfile.py`.

The `LoadUser` class represents simulated users accessing the product listing endpoint.

The configuration includes:

- Wait intervals between 5 and 15 seconds.
- Product listing requests.
- HTTP response validation.
- Success and failure reporting.

The load-testing script allows the application to be evaluated under concurrent request activity.

---

9. Testing and Quality Assurance

9.1 Testing Strategy

Pytest is used to verify the functionality and correctness of the application.

The tests are separated into two modules:

- `test_products.py`
- `test_cache.py`

The product API tests use an in-memory SQLite database to isolate test data from the application's development database.

The tests also override the application's database dependency during execution.

9.2 API Testing

The product test suite covers:

Test	Purpose
test_read_root	Verify the root endpoint
test_get_products	Verify product listing and retrieval
test_add_product	Verify creation and input validation
test_read_product	Verify individual product retrieval

test_update_product	Verify partial updates and validation
test_delete_product	Verify product deletion

The tests include checks for:

- Successful HTTP responses.
- Invalid request data.
- Product creation.
- Product retrieval.
- Product modification.
- Missing product handling.
- Product deletion.
- Database state after invalid requests.

9.3 Cache Testing

The cache test suite covers:

Test	**Purpose**
test_cache	Verify setting and retrieving cache entries
test_del_cache	Verify individual cache deletion
test_clear_cache	Verify complete cache clearing

9.4 Test Database Isolation

The product tests use an in-memory SQLite database.

The test fixture:

1. Clears the application cache.
2. Creates database tables.
3. Overrides the database dependency.

4. Starts the FastAPI TestClient.
5. Executes the test.
6. Removes the dependency override.
7. Drops the test tables.
8. Clears the cache again.

This approach prevents test data from affecting the development database.

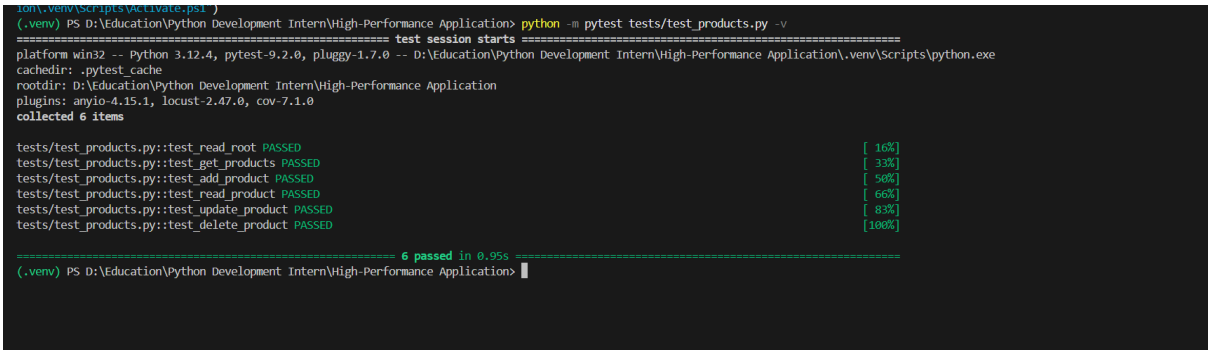


Figure 5 Automated testing results

---

10. Performance Evaluation

10.1 Performance Testing Objective

Performance testing is used to evaluate how the application responds when multiple users access its API.

Locust was selected to simulate concurrent users and collect request statistics.

The evaluation focuses on:

- Number of requests.
- Failed requests.
- Average response time.
- Median response time.
- 95th and 99th percentile response times.
- Minimum and maximum response times.
- Requests per second.

10.2 Locust Benchmark Results

The following results were recorded during a Locust test with 30 simulated users.

Metric	Observed Result
Simulated users	30
Total requests	509
Failed requests	0
Average response time	8.09 ms
Median response time	8 ms
95th percentile	11 ms
99th percentile	20 ms
Minimum response time	3 ms
Maximum response time	111 ms
Average response size	1,176 bytes
Reported current RPS	3.3

10.3 Result Analysis

The recorded test produced 509 successful requests without any reported failures.

The median response time was 8 milliseconds, while the 95th percentile was 11 milliseconds.

The 99th percentile was 20 milliseconds, indicating that most requests completed within a relatively short response time during this particular test.

The maximum observed response time was 111 milliseconds.

These results demonstrate that the application handled the recorded test workload successfully.

However, the results should not be interpreted as proof of production-scale capacity.

The benchmark represents a specific test environment and workload. Hardware specifications, test duration, cache state, and network conditions should be documented when conducting repeatable comparisons.



Figure 6 Locust performance results

10.4 Performance Optimization Techniques

The application implements several techniques intended to improve response efficiency.

In-memory caching: Reduces repeated database retrieval for cached requests.

Pagination: Restricts the number of product records retrieved in an individual listing request.

Primary-key retrieval: Uses the product primary key for individual record lookups.

Cache invalidation: Prevents outdated cached information from being reused after product modifications.

Request validation: Rejects invalid requests before they proceed through the product management operations.

---

11. Application Logging and Monitoring

11.1 Logging Configuration

The application uses Python's built-in logging module.

A shared logging configuration is maintained in `configs/general_logger.py`.

The `get_logger()` function creates named loggers and associates them with module-specific log files.

The log formatter records:

- Timestamp.
- Severity level.
- Logger name.
- Log message.

11.2 Log Organization

The current application uses two log files:

Log File	Responsibility
main_api.log	API application events
memory_cache.log	Cache-related events

This separation makes it easier to identify whether an event originated from the API or caching module.

11.3 Monitoring Approach

The current monitoring approach consists of:

- Application log files.
- Locust request statistics.
- Response-time measurements.
- Request failure counts.
- Python profiling through cProfile.

These tools provide basic visibility into application behavior and performance.

A centralized monitoring dashboard and continuous production monitoring system have not been implemented.

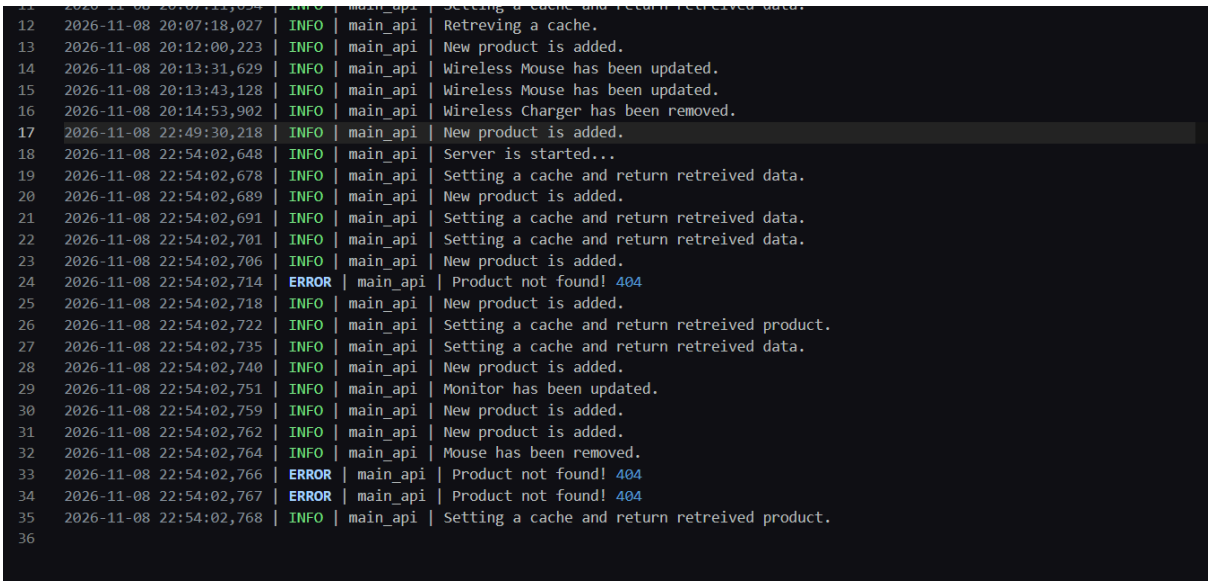


Figure 7 Application logging output

---

12. Installation and Execution

12.1 Prerequisites

The following software is required:

- Python 3.12 or compatible version.
- pip package installer.
- Visual Studio Code or another Python development environment.
- Git (optional, for repository management).

12.2 Environment Setup

Clone or download the project repository.

Open the project directory in the terminal.

Create a virtual environment:

```shell
python -m venv .venv
```

Activate the virtual environment on Windows:

```
Shell
.venv\Scripts\activate
```

Install the project dependencies:

```
Shell
python -m pip install -r requirements.txt
```

12.3 Running the Application

Start the FastAPI application using Uvicorn:

```
Shell
uvicorn main:app --reload
```

The application will be available at:

```
None
http://127.0.0.1:8000
```

Interactive API documentation:

```
None
http://127.0.0.1:8000/docs
```

12.4 Running Automated Tests

Execute the test suite using:

```
Shell
python -m pytest -v
```

To generate a coverage report:

```shell
python -m pytest --cov=. --cov-report=term-missing
```

12.5 Running Load Tests

With the application running, open another terminal and execute:

```shell
locust -f locustfile.py --host http://127.0.0.1:8000
```

Locust provides an interface for configuring the number of simulated users and the request spawning rate.

The application must remain running while load testing is performed.

---

13. Scalability Considerations and Future Improvements

The current application provides a foundation for understanding performance-oriented web application design.

However, several improvements could be considered before using the application in a larger production environment.

13.1 Database Scalability

SQLite is appropriate for lightweight applications and development environments.

For larger deployments involving concurrent database writes and multiple application instances, PostgreSQL could provide a more suitable database architecture.

Additional indexes could also be introduced based on actual query patterns and profiling results.

13.2 Distributed Caching

The current cache exists in the memory of the running application process.

If multiple application instances are introduced, each instance would maintain its own independent cache.

Redis could be considered as a centralized caching solution to share cached data between instances.

13.3 Asynchronous Database Operations

The current application uses synchronous SQLAlchemy sessions.

An asynchronous database architecture could be explored when the workload requires efficient handling of high numbers of concurrent I/O operations.

This would require changes to database configuration, session management, and endpoint implementation.

13.4 Containerization

Docker could be introduced to package the application and its dependencies into a consistent deployment environment.

This would simplify application deployment across different systems.

13.5 Orchestration

Kubernetes could be considered for managing multiple application instances, deployment replicas, service discovery, and scaling.

It is a potential future deployment approach rather than part of the current implementation.

13.6 Advanced Monitoring

Prometheus and Grafana could be introduced to collect and visualize application metrics.

Useful metrics would include:

- Request rate.
- Request latency.
- HTTP error rate.
- CPU and memory usage.
- Database response time.
- Cache hit and miss rates.

13.7 Scalability Limitations

The current application has the following limitations:

- SQLite is used instead of a production-oriented database server.
- Caching is limited to one application process.
- No distributed cache is implemented.
- No dedicated asynchronous database operations are implemented.
- No Docker or Kubernetes deployment configuration is included.
- No centralized monitoring platform is implemented.
- The available Locust benchmark covers a limited test workload.

These limitations provide clear directions for future development.

---

14. Conclusion

The High-Performance Application with Advanced Architecture demonstrates the implementation of a product management API with an emphasis on performance optimization, database efficiency, caching, testing, and application monitoring.

The project combines FastAPI, SQLAlchemy, SQLite, Pydantic, and cachetools to provide a structured application architecture.

Pagination reduces unnecessary record retrieval, while in-memory caching reduces repeated database access for frequently requested data. Cache synchronization and invalidation help maintain consistent application behavior.

Automated testing verifies important API operations, validation rules, and cache functionality. Locust provides performance measurements under simulated concurrent user activity.

The recorded benchmark demonstrated successful handling of 509 requests with no reported failures during the tested workload.

Although the application does not yet include distributed infrastructure or production-scale monitoring, it provides a practical foundation for understanding how performance-oriented web services are designed, implemented, tested, and evaluated.

The project also establishes a foundation for future exploration of asynchronous database operations, distributed caching, containerization, and scalable deployment architectures.